MUHAMMAD TAHA
22F-3277
BCS-3C
22/09/2023

LAB 5

(Total Marks 15)

Task 1: Draw and run following code and write description of execution.

```
org 0x100
 jmp start

multiplicand: db 13
multiplier: db 5
result: db 0

start: mov cl, 4
       mov bl, [multiplicand]
       mov dl, [multiplier]

checkbit: shr dl, 1
         jnc skip
         add [result], bl

skip: shl bl, 1
      dec cl
      jnz checkbit

mov ax, 0x4c00
int 0x21
```

Also fill the following table after execution of each loop in binary form.

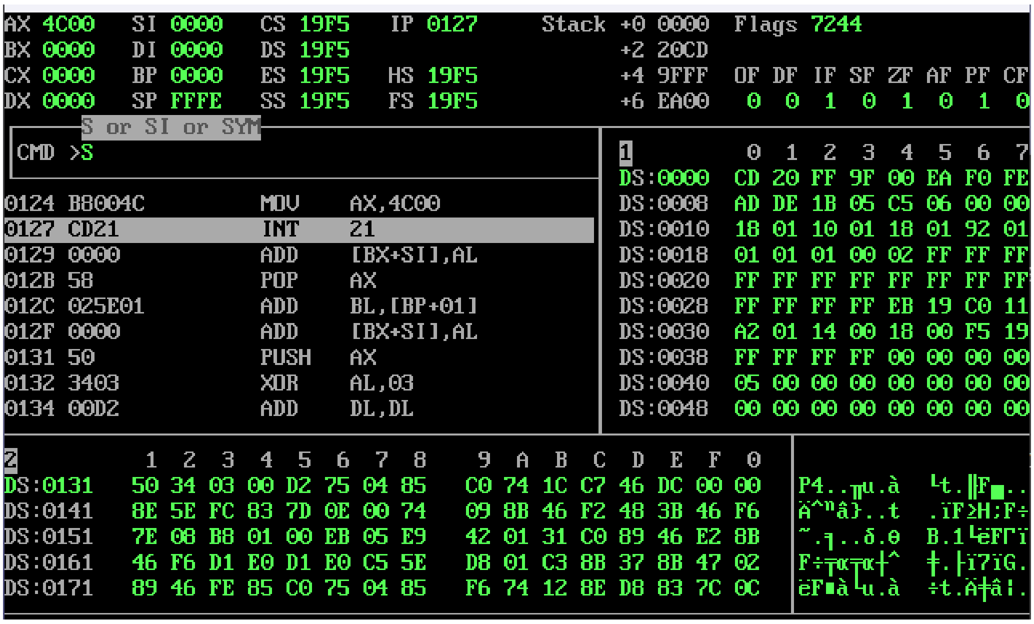
Count	Multiplicand	Multiplier	Result
4	11010	0010	1101
3	110100	0001	1101
2	1101000	0000	01000001
1	11010000	0000	01000001

We take the first digit of the multiplier and multiply it with the multiplicand. If the digit of multiplier is one the answer will be multiplicand itself. So, it is placed in result. Before multiplying with the next digit, as we do in common multiplication a cross is placed at the right most place on next line and the result is shifted to left, so in this we now must shift multiplicand to the left so we will use left shift. Now we again check the multiplier bit and if it is zero perform no addition. Left shift the multiplicand before the next bit of multiplier is tested. And same as this then we add and place it in result and for next we will again shift it to left as in simple multiplication we place two crosses so here it will be second shift so this process is repeated and until final result.

Task 2: Write assembly language code to multiply two-word size numbers:

- **Multiply 600* 350**
- **Use shift right, shift left and rotate carry left operations**

```
org 0x0100

mov cl, 16
mov dx, [multiplier]

checkbit:
   shr dx, 1
   jnc skip
   mov ax, [multiplicand]
   add [result], ax
   mov ax, [multiplicand+2]
   adc [result+2], ax

skip:
   shl word [multiplicand], 1
   rcl word [multiplicand+2], 1
   dec cl
   jnz checkbit

mov ax, 0x4c00
int 0x21

multiplicand: dd 600
multiplier: dd 350
result: dd 0
```



Task 3: Write a program to count the number of 1’s in a word size four variable in single label.

- **num: dw 65535, 65535, 39321, 39321**
- **Use Rotate Left (ROL) Operation**
- **Count the numbers of 1’s and save it into another variable with its address.**

```
org 0x0100

mov ch,4
mov si, num
mov bx,0

outer:
        mov ax, [si]
        mov cl, 16

checkone:
        rol ax, 1
        jnc skip
        add bx, 1

skip:
        dec cl
        jnz checkone

        add si, 2
        dec ch
        jnz outer

mov [onescount], bx

mov ax,0x4c00
int 0x21

num: dw 65535,65535,39321,39321
onescount: dw 0
```